

Unidade Central de Processamento

Simulação



Simulador - Código Máquina

Data Movement Instructions:

Assembly Language Instruction:	Example:	Meaning:	Machine Language Instruction:
LOAD [REG] [MEM]	LOAD R2 13	$R2 = M[13]$	1 000 0001 0 RR MMMMM
STORE [MEM] [REG]	STORE 8 R3	$M[8] = R3$	1 000 0010 0 RR MMMMM
MOVE [REG1] [REG2]	MOVE R2 R0	$R2 = R0$	1 001 0001 0000 RR RR

Arithmetic and Logic Instructions:

Instruction:	Example:	Meaning:	Machine Language Instruction:
ADD [REG1] [REG2] [REG3]	ADD R3 R2 R1	$R3 = R2 + R1$	1 010 0001 00 RR RR RR
SUB [REG1] [REG2] [REG3]	SUB R3 R1 R0	$R3 = R1 - R0$	1 010 0010 00 RR RR RR
AND [REG1] [REG2] [REG3]	AND R0 R3 R1	$R0 = R3 \& R1$	1 010 0011 00 RR RR RR
OR [REG1] [REG2] [REG3]	OR R2 R2 R3	$R2 = R2 R3$	1 010 0100 00 RR RR RR

http://www.assis.pro.br/public_html/davereed/14-DentroDoComputador.html

Simulador - Código Máquina

Branching Instructions:

Instruction:	Example:	Meaning:	Machine Language Instruction:
BRANCH [MEM] BZERO [MEM] BNEG [MEM]	BRANCH 10 BZERO 2 BNEG 7	PC = 10 PC = 2 IF ALU RESULT IS ZERO PC = 7 IF ALU RESULT IS NEGATIVE	0 000 0001 000 MMMMM 0 000 0010 000 MMMMM 0 000 0011 000 MMMMM

Other Instructions:

Instruction:	Example:	Meaning:	Machine Language Instruction:
NOP HALT	NOP HALT	Do nothing. Halt the machine.	0000 0000 0000 0000 1111 1111 1111 1111

Simulador - Programa exemplo

Significado	Assembler	Linguagem Máquina	Comentário
$R0 = M[5]$	LOAD R0 5	1000000100000101	// carrega posição de memória 5 em R0
$R1 = M[6]$	LOAD R1 6	1000000100100110	// carrega posição de memória 6 em R1
$R2 = R0 + R1$	ADD R2 R0 R1	1010000100100001	// adiciona R0 e R1, armazena em R2
$M[7] = R2$	STORE 7 R2	1000001001000111	// armazena R2 na posição de memória 7
HALT	HALT	1111111111111111	// Pára

Simulador - Programa exemplo

As primeiras cinco posições de memória (endereços de 0 até 4) contém instruções em linguagem de máquina para somar dois números e armazenar sua soma de volta na memória. Os números podem a serem somados são armazenados na posições de memória 5 e 6. Para executar este programa, a Unidade de Controle deve seguir os seguintes passos:

Primeiro, o contador de programa é inicializado: $PC = 0$.

É feita a aquisição da instrução na posição de memória 0 (correspondente ao valor actual do PC) e o PC é incrementado: $PC = 0 + 1 = 1$. CPU é configurada de modo que o conteúdo da posição de memória 5 seja carregado no registo R0 e um ciclo de CPU é executado.

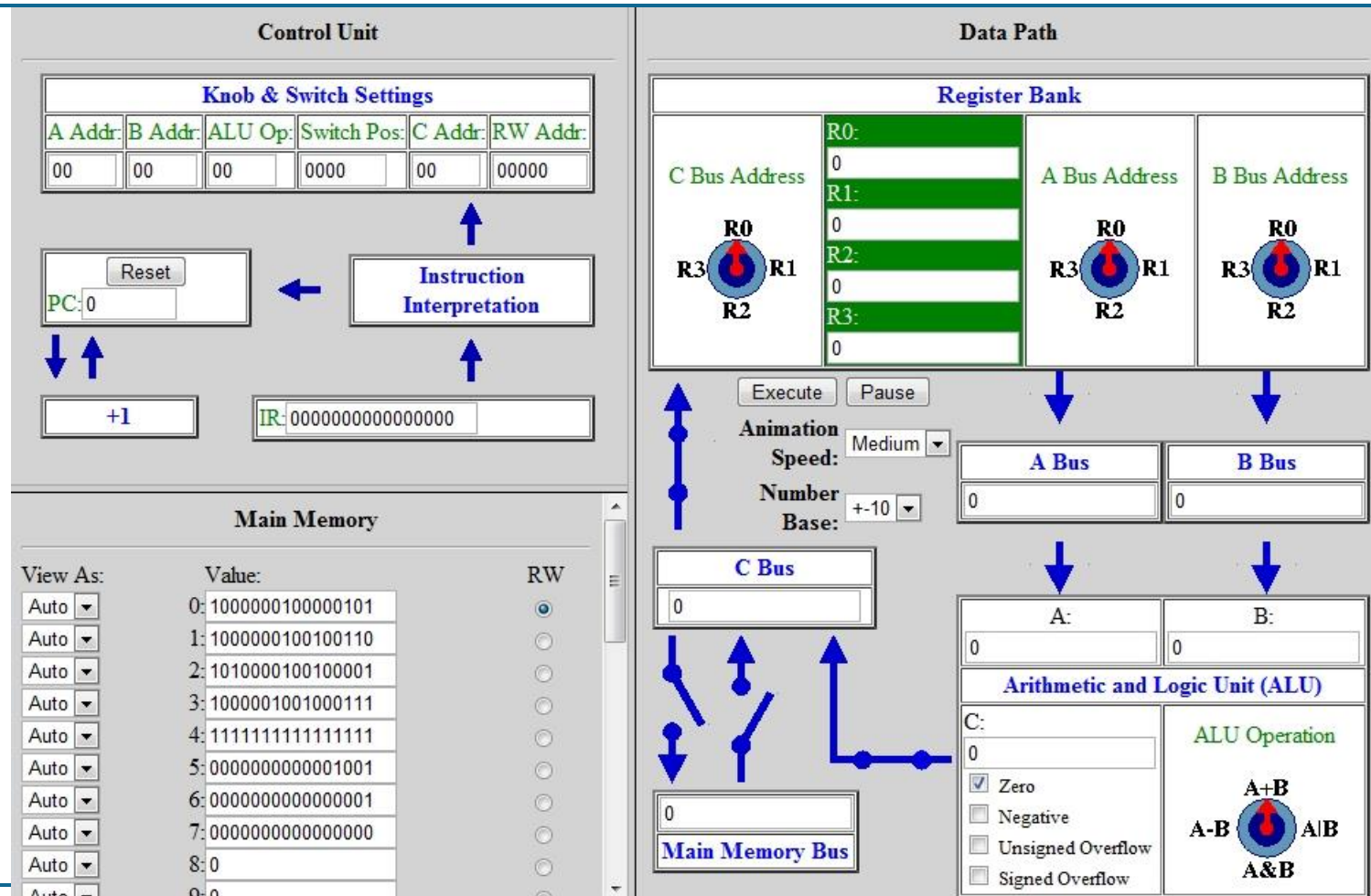
É feita a aquisição da instrução na posição de memória 1 (correspondente ao valor actual do PC) e o PC é incrementado: $PC = 1 + 1 = 2$. Como esta instrução (1000000100100110) não é uma instrução HALT, ela é decodificada: o hardware da CPU é configurado de modo que o conteúdo da posição de memória 6 seja carregado no registo R1 e um ciclo de CPU é executado.

A próxima instrução, na posição de memória 2 (correspondente ao valor actual do PC) é adquirida e o PC é incrementado: $PC = 2 + 1 = 3$. Como esta instrução (1010000100100001) não é uma instrução HALT, ela é decodificada: o hardware da CPU é configurado de modo que o conteúdo do registo R0 é somado ao conteúdo do registo R1 e o resultado é armazenado no registo R2 e um ciclo de CPU é executado.

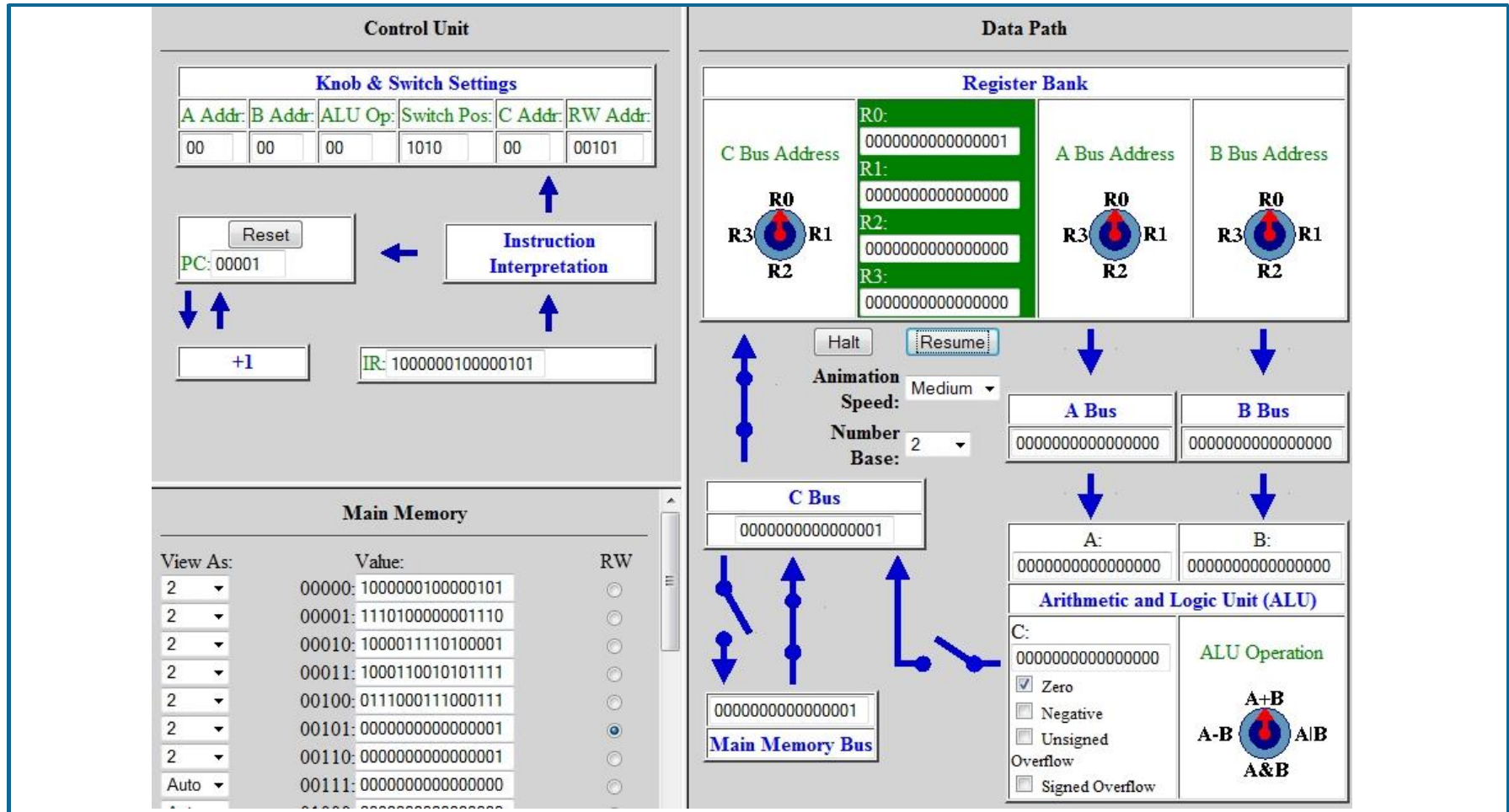
A próxima instrução, na posição de memória 3 (correspondente ao valor actual do PC) é adquirida e o PC é incrementado: $PC = 3 + 1 = 4$. Como esta instrução (1000001001000111) não é uma instrução HALT, ela é decodificada: o hardware da CPU é configurado de modo que o conteúdo do registo R2 seja armazenado na posição de memória 7 e um ciclo de CPU é executado.

A próxima instrução, na posição de memória 4 (correspondente ao valor actual do PC) é adquirida e o PC é incrementado: $PC = 4 + 1 = 5$. Como esta instrução (1111111111111111) é uma instrução HALT, a Unidade de Controle reconhece o fim do programa e para a execução.

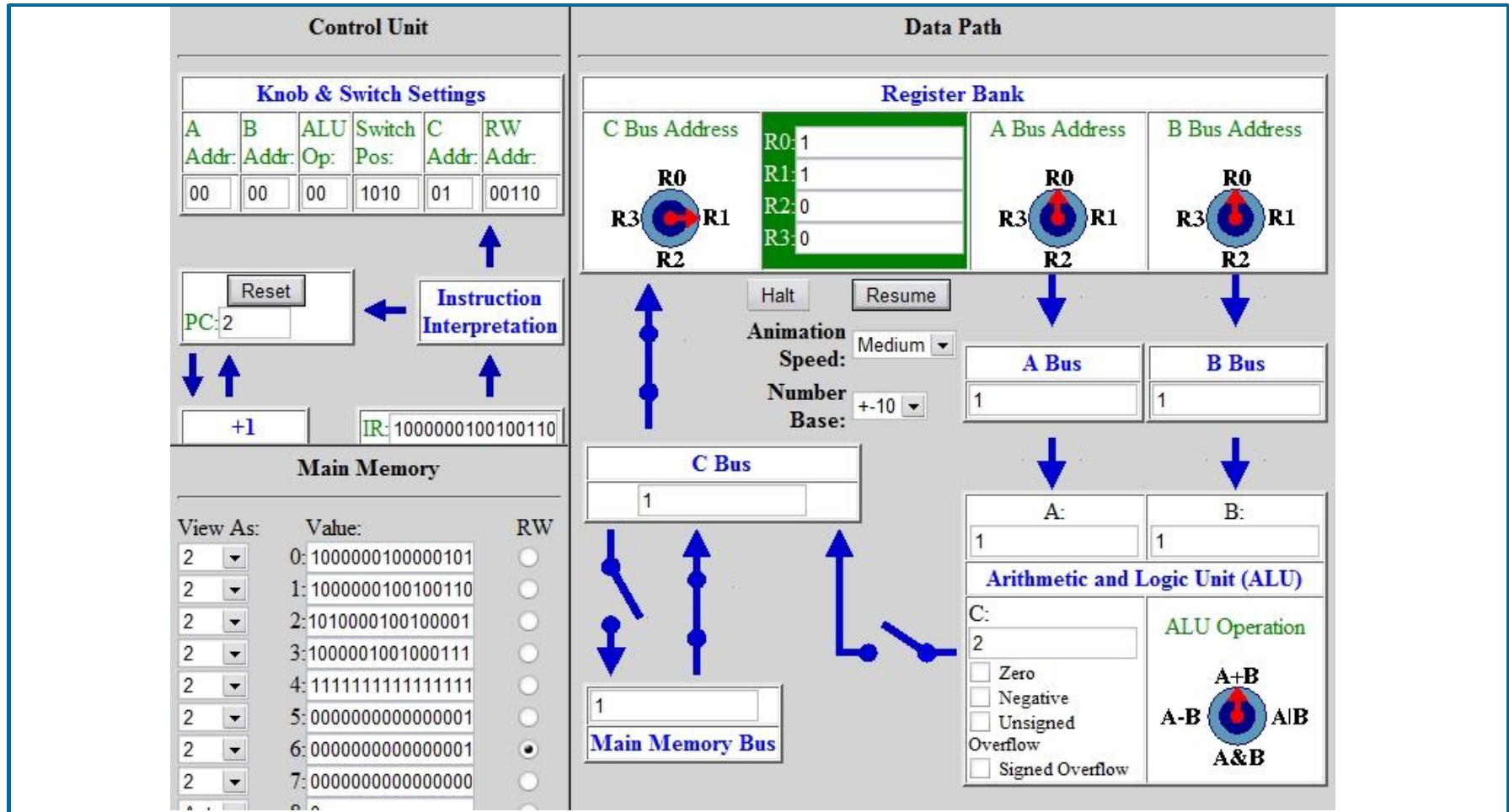
Estado inicial do simulador



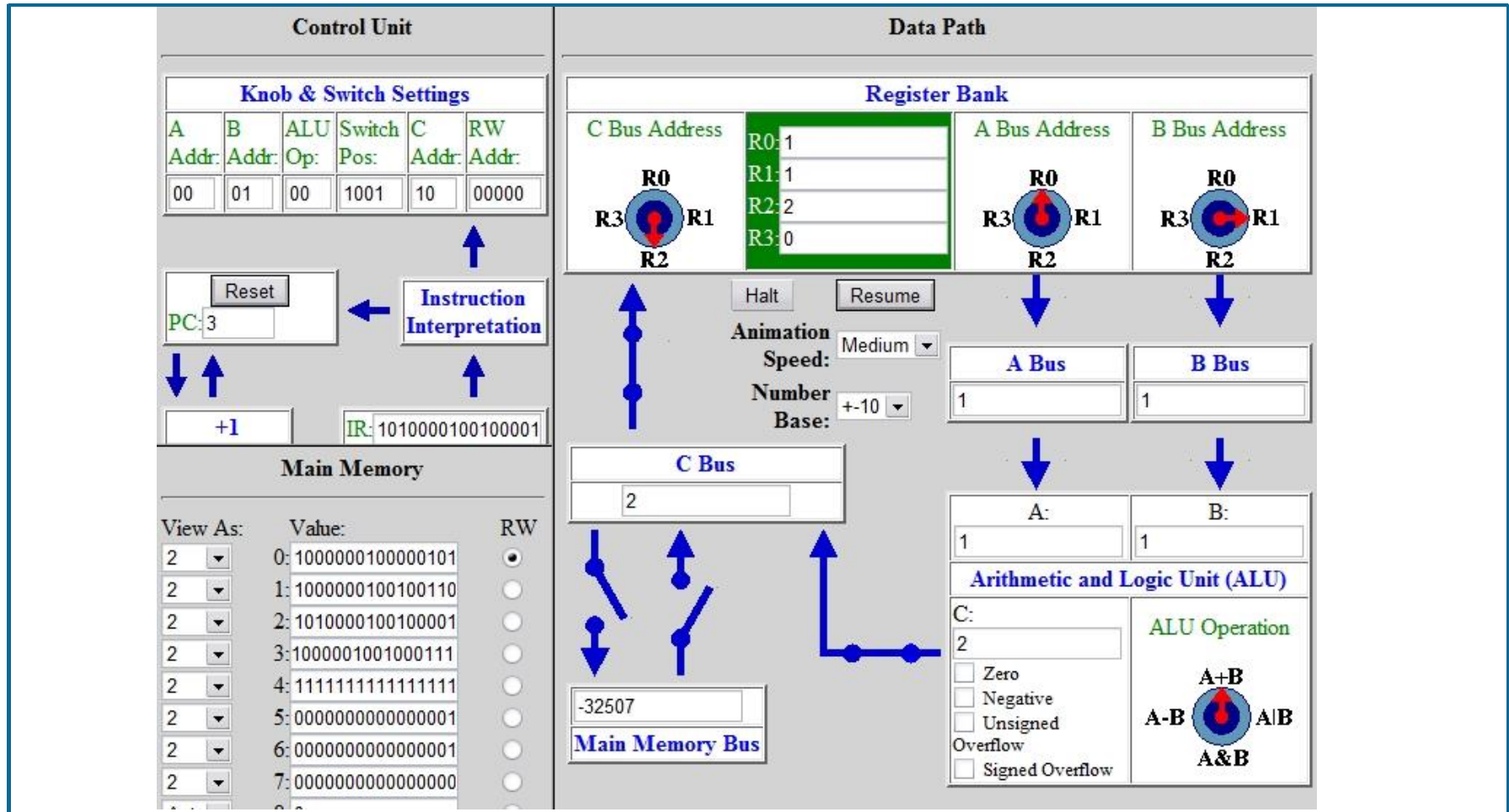
Simulador depois da primeira instrução (R0 = MM5)



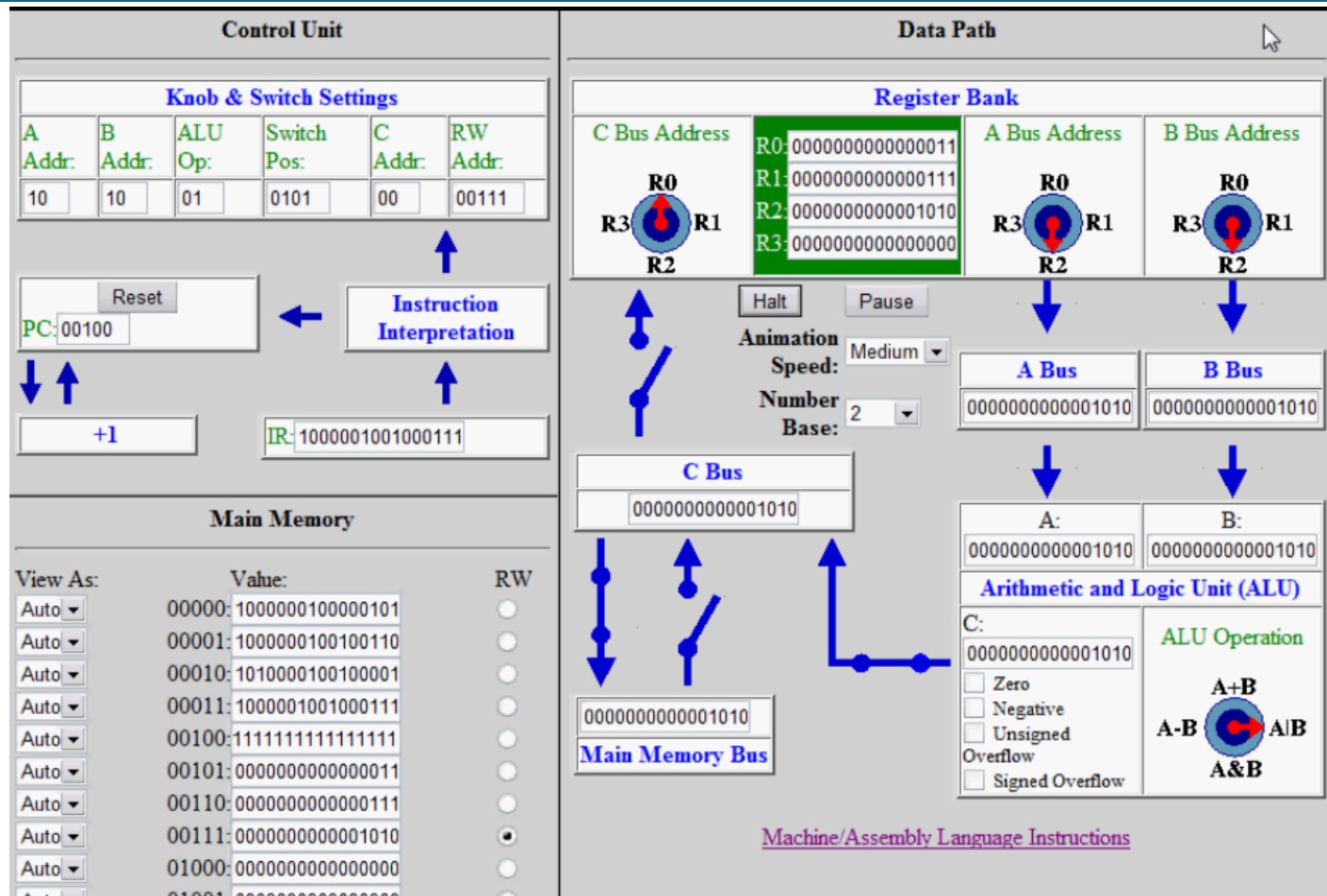
Simulador depois da segunda instrução (R1 = MM6)



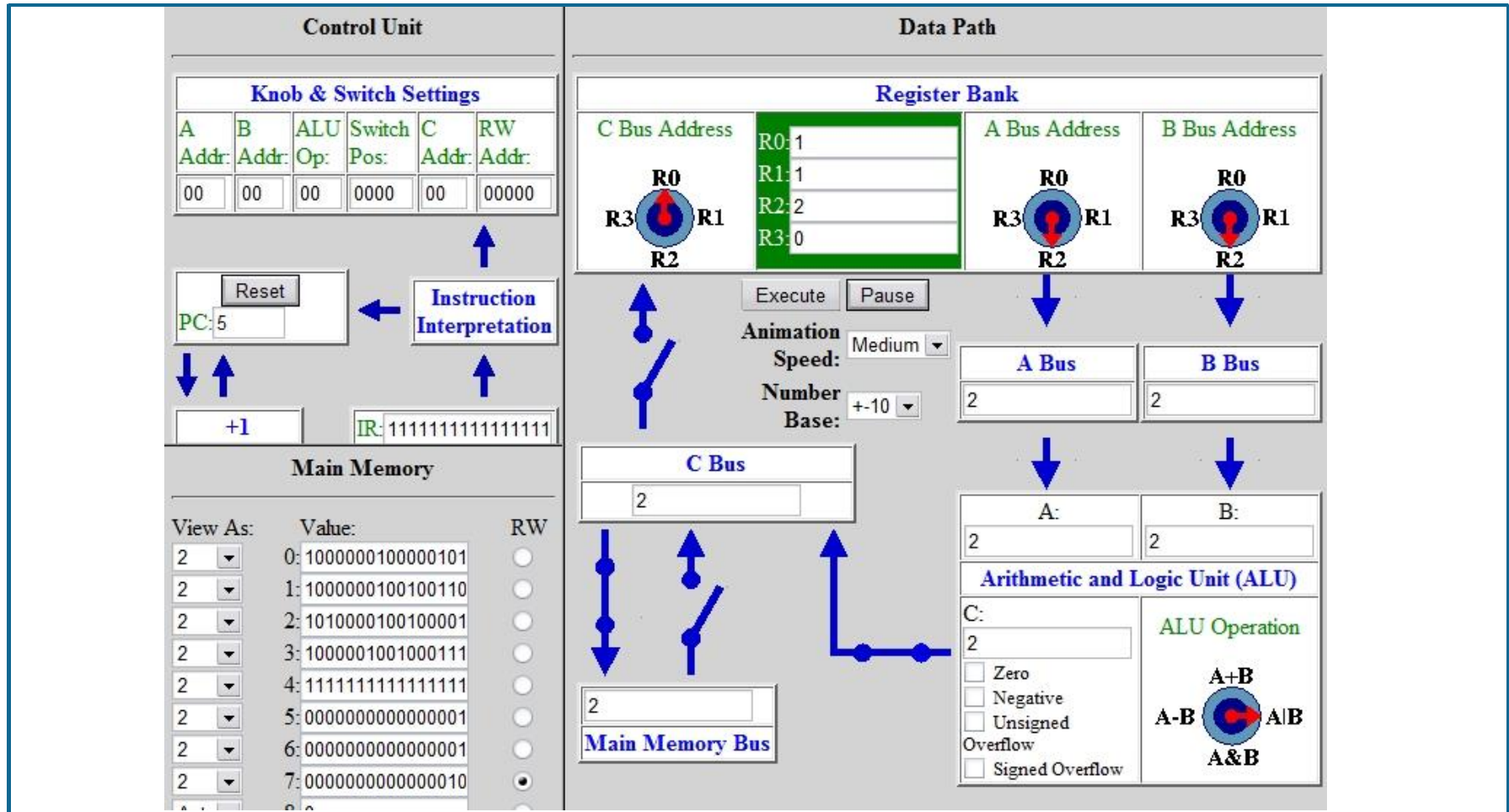
Simulador depois da terceira instrução ($R2 = R0 + R1$)



Simulador depois da quarta instrução (MM7 = R2)



Simulador depois da quinta instrução (HALT).



Simulador - Programa exemplo

```
0: LOAD R0 5    // R0 = M[5]
1: LOAD R1 6    // R1 = M[6]
2: ADD R2 R1 R0 // R2 = R1 + R0
4: STORE 7 R2   // M[7] = R2
5: HALT        // Halt the machine.
```

```
0: 1000000100000101 // carrega posição de memória 5 em R0
1: 1000000100100110 // carrega posição de memória 6 em R1
2: 1010000100100001 // adiciona R0 e R1, armazena em R2
3: 1000001001000111 // armazena R2 na posição de memória 7
4: 1111111111111111 // parada
5: 00000000000001001 // dado a ser somado: 9
6: 00000000000000001 // dado a ser somado: 1
7: 0000000000000000 // posição onde a soma é armazenada
```